Agentic dev tools events: the Postman Agent Stack and scoring rubric

How `events-agenttools` decides which developer events are worth a DevRel presence, and why. The skill itself (`skills/events-agenttools/SKILL.md`) just runs the searches and writes the scorecard — the actual judgment calls live here and in `skills/events-agenttools/references/scoring.md`, which is what the skill reads at run time. This doc is the human-readable version of that reference file.

Why this exists

postman-plugin doesn't have one target developer — it's the entry point into a set of Postman products that all serve **AI builders**, but at different points in an agent's lifecycle. Treating "postman-plugin adoption" as a single audience would miss most of the events worth attending. Splitting it into concerns makes the scoring legible: an event can be a strong fit for one concern and irrelevant to another.

The three concerns

Concern	Question the developer is asking	Products
Building	How do I build an app/agent that calls APIs autonomously?	postman-plugin (specs/collections), Orbit (API discovery)
Maintaining	How do I keep the APIs my agents depend on from breaking?	postman-plugin (mocking/testing), Context Graph (dependency/blast-radius graph)
Operating	How do I run, govern, and secure the agents themselves once they're live?	Fabric Gateway (routing/policy/observability), Astropods (registry/budget/audit), Passport (credential/identity security)

postman-plugin is the only product that spans two concerns — Building and Maintaining — because it runs inside the agent tool itself (Claude Code, Cursor, Codex, Kiro, Antigravity, and others) at both stages: writing a new spec and mocking/testing an existing one are the same filesystem-first workflow, just pointed at different targets. Both stages belong to the same persona, API/Platform Engineers — the people who use the coding agent to do the building and the maintaining, not a separate audience.

The six products

Product	Concern	Role
postman-plugin	Building + Maintaining	Filesystem-first, git-native API dev inside coding agents
Orbit	Building	Free API discovery for agents over REST/MCP
Context Graph	Maintaining	Dependency graph — "what breaks if I change this endpoint?"
Fabric Gateway	Operating	Agent gateway — routing, policy, fallbacks, credential centralization
Astropods	Operating	Governance/ops for a fleet of agents — registry, budget limits, audit trail — and the runtime for running those agents on a schedule or fully autonomously (managed agents in the cloud)

Passport	Operating	Secure API access for agents — credential references instead of exposed API keys, durable/ephemeral agent identities

Persona labels

Every persona works inside or alongside an agentic coding tool (Claude Code, Cursor, Codex, Kiro, Antigravity, Factory Droid, Kimi Code, OpenCode, Pi, or similar) — that thread runs through all four, not just one bucket:

- **Agentic App Builders** — design and build apps where agents call APIs autonomously, including discovering/evaluating the right API (Orbit)
- **Headless Agents** — build and operate agents that run without a human in the loop
- **API/Platform Engineers** — own the APIs and production systems agents call, and use a coding agent (Claude Code, Cursor, Codex, Kiro, Antigravity, or similar) as the tool to both build and maintain them; map to postman-plugin (building + maintaining) and Context Graph
- **Agent Ops Engineers** — responsible for the agents themselves in production — evals, token/cost, reliability, observability, credential security, governance; map to Fabric Gateway, Astropods, and Passport

Scoring rubric (0–100)

Factor	Weight	What it measures
Agentic-tool audience fit	30	Known users of Claude Code, Cursor, Codex, Kiro, Antigravity, or similar — or the API/platform/agent-ops engineers who support them
Headless/agent-workflow relevance	25	Does the event cover agentic app building, autonomous agents calling APIs, agent evals/observability/cost, agent governance, credential/identity security, or CLI-first/git-native dev workflows
PLG format signal	20	Hands-on, individual-developer-led (hackathon, workshop, meetup) vs. buyer-facing (analyst conference, exec summit)
Reach & timing	15	Attendee count, 2026 relevance, whether a DevRel presence is realistically available
Topic/demo opportunity	10	Is there a natural slot for a live demo rather than just a booth

Score bands: 80–100 strong fit, 60–79 good fit, 40–59 moderate fit, 20–39 weak fit, 0–19 poor fit. Full band descriptions and the presence-type recommendation table (hackathon sponsor / talk / workshop / demo table / booth / pass) live in `skills/events-agenttools/references/scoring.md`, since that's the copy the skill actually reads at run time — this doc won't be kept in sync line-for-line if the rubric changes, treat the reference file as the source of truth for exact wording.

Output

Each run writes `events-agenttools-output/events-agenttools.md`: a scored table sorted highest-first, a "Top Picks" section with the top 3 and why, and a "Pass" section for anything scoring below 40.